Instantané du 29 octobre 2026. Ce dossier décrit les quatre dépôts tels qu'ils étaient ce jour-là ; les projets ont continué d'avancer depuis, et les dépôts font foi. Les liens soulignés mènent aux sources, figées à leur état de ce jour. La suite vit sur gnu6dotlive.github.io.

DOSSIER 2 · LE MOTEUR

gnostral.rs

Un laboratoire ouvert qui cherche à faire tourner de gros modèles d'intelligence artificielle sur une carte graphique modeste — et qui publie ses mesures, y compris celles qui déçoivent.

RECHERCHE · V0.8.0 · LICENCE MIT

2,12×
plus vite que la version précédente du **même moteur**

20,18
jetons par seconde, contre 9,51 au départ

22 → 13
couches placées sur la carte selon la mémoire libre

3/3
familles de modèles servies par le même exécutable

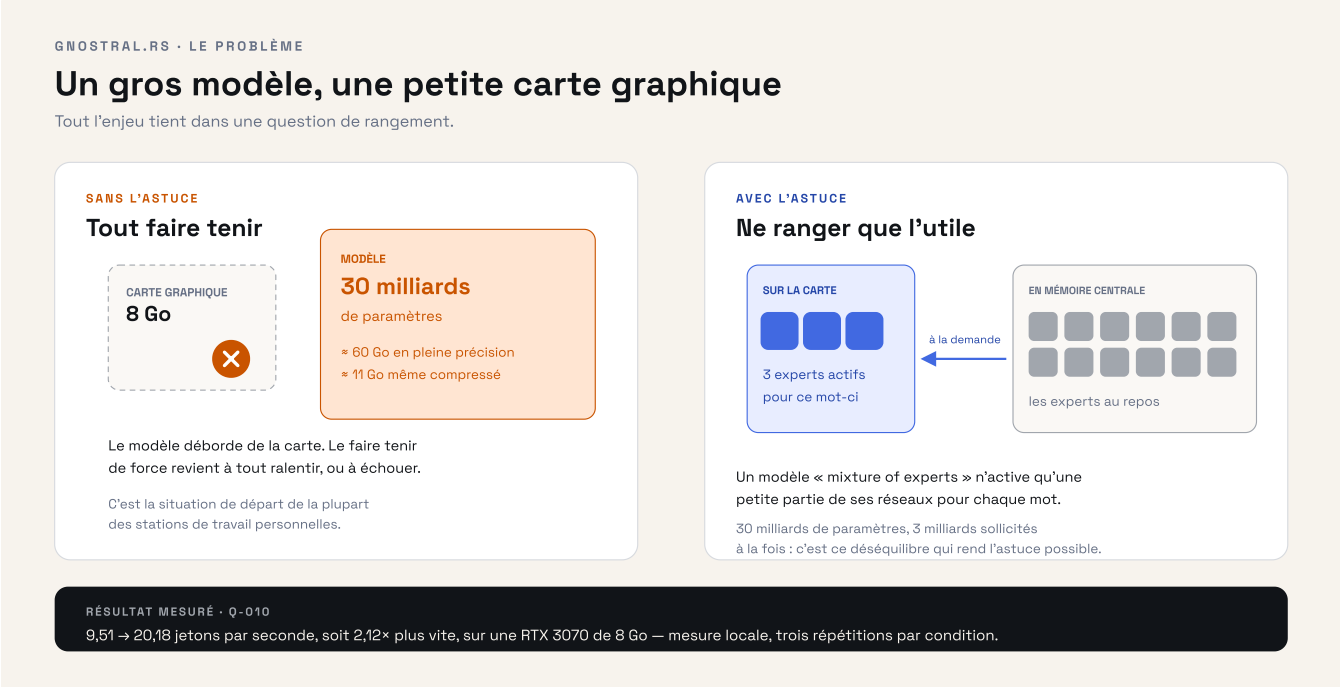
En une image

L'ANALOGIE

Imaginez un entrepôt et un petit comptoir. Le **comptoir**, c'est la carte graphique : étroit, mais très rapide. L'**entrepôt**, c'est la mémoire centrale : immense, mais lent à desservir.

Un modèle de 30 milliards de paramètres ne tient pas sur le comptoir. L'astuce consiste à n'y poser que les trois milliards de morceaux utiles à l'instant présent, et à aller chercher les autres à l'entrepôt seulement quand ils servent. C'est tout l'enjeu de gnostral.rs.

Le problème, puis l'astuce



Un problème de rangement. Un modèle « mixture of experts » n'active qu'une petite partie de ses réseaux pour chaque mot. Ce déséquilibre rend le découpage possible.

La machine de référence n'est pas un serveur de datacenter : une RTX 3070 de 8 Go de mémoire graphique, 32 Go de mémoire centrale. C'est une configuration de station de travail personnelle — et c'est précisément ce qui rend la recherche intéressante.

Le résultat principal : en gardant sur la carte les parties toujours sollicitées et en empruntant les experts quantifiés depuis la mémoire centrale, le débit passe de **9,51 à 20,18 jetons par seconde**, soit 2,12 fois plus vite. Mesuré localement, trois répétitions par condition, réponses identiques octet pour octet.



La même réponse, au rythme mesuré. Les deux couloirs écrivent exactement le même texte — c'est vérifié octet pour octet. Sans le correctif, la lecture de la question dure 7,2 s, puis 9,5 jetons par seconde ; avec, 2,7 s puis 20,2 jetons par seconde. Le temps affiché est le temps réel.

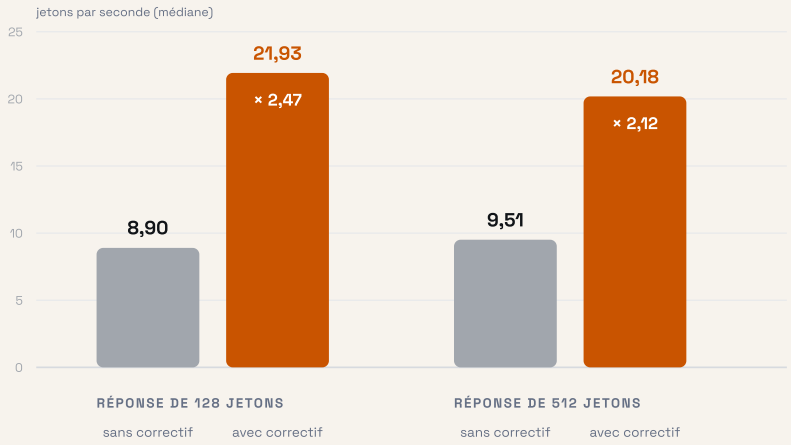
Ce qui a été mesuré — et dans quel cadre

JALON	RÉSULTAT	CE QUE CELA NE DIT PAS
Experts empruntés Q-010	9,51 → 20,18 jetons/s sur une réponse de 512 jetons, médiane de trois répétitions, 2,12×	Pas plus rapide que llama.cpp (34,54 jetons/s sur la même machine) ; pas valable pour toutes les architectures.
Placement au chargement Q-011	22/21/21 couches sur la carte → 13/12/13 sous réservation de mémoire	Placement initial seulement ; aucune migration pendant le service.
Un moteur, trois familles Q-012	3/3 : modèle dense, embeddings, MoE quantifié	Trois serveurs successifs ; pas de co-résidence, pas de stabilité de production.
Vrai processus dense Q-013-C	Réponse sémantique exacte « GNOSTRAL » ; +8 Mio de mémoire GPU après arrêt	Un seul modèle, un serveur de courte durée, pas une API publique.

GNOSTRAL.RS · MESURE A/B

Deux fois plus vite, à mémoire presque égale

Même machine, même modèle, même binaire — sauf le correctif mesuré. Trois répétitions par condition.



CE QUE LA VITESSE A COÛTÉ

Premier mot affiché
3 818 → 1 422 ms

Pré-remplissage, 512 jetons
7 248 → 2 676 ms

Mémoire GPU totale, au pic
2 344 → 2 351 Mio

Mémoire hôte, au pic
22 094 → 22 172 Mio

Ce gain est relatif au même moteur sans le correctif. Face à llama.cpp, gnostral atteint environ 58 % du débit — l'écart reste ouvert.

Données : gnostral.rs/evidence/runs/q010-hotpath-ssd-ab-audit-20261010.json — 12 réponses mesurées, 3 processus par condition.

Le gain, et son prix. Même machine, même modèle, même binaire sauf le correctif : le débit double, le premier mot arrive 2,7 fois plus tôt, et la mémoire graphique ne bouge que de 7 Mio. La référence reste le même moteur sans correctif — pas llama.cpp.

GNOSTRAL.RS · PLACEMENT SOUS PRESSION

Quand la mémoire se remplit, le modèle se replie

Le moteur place chaque couche sur la carte ou sur le processeur, selon la mémoire qu'on lui laisse.

48 COUCHES · CONTRÔLE



22 sur la carte
26 sur le processeur

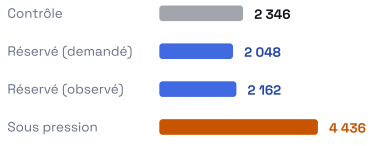
48 COUCHES · SOUS PRESSION MÉMOIRE



13 sur la carte
35 sur le processeur

Débit de décodage : **20,88** → **20,15** jetons/s (−3,5 %)

MÉMOIRE GPU TOTALE, AU PIC



Mio · échelle 0 – 5 000

La mémoire « totale » inclut le bureau.

CE QUI EST QUALIFIÉ — ET CE QUI NE L'EST PAS

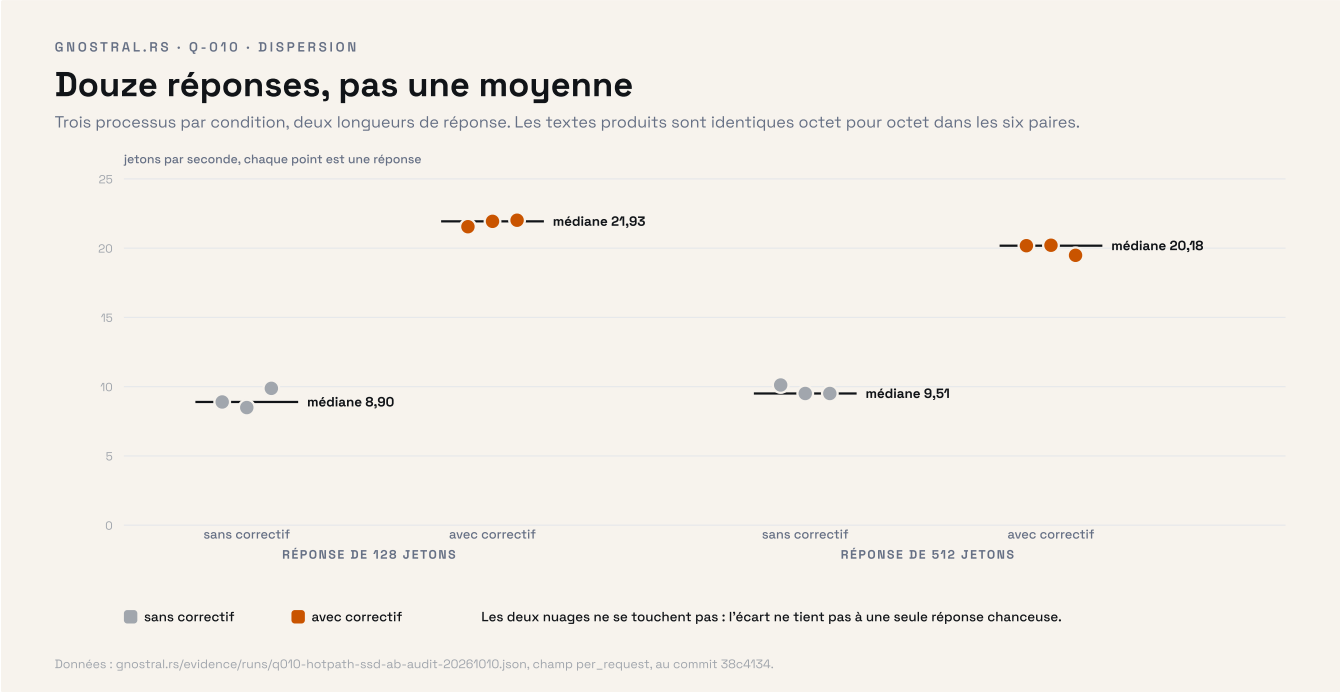
Le placement initial selon la mémoire disponible est observé. Le rééquilibrage en cours d'exécution, lui, reste non qualifié : le moteur ne déplace pas les couches après le démarrage.

Données : gnostral.rs/evidence/runs/q011-rtx3070-placement-observation-20261010.json — 3 répétitions par condition, RTX 3070 8 Gio.

Le placement, observé. Quand on occupe de la mémoire graphique avant le démarrage, le moteur pose moins de couches sur la carte (22 → 13 dans ces deux essais) et le débit ne recule que de 3,5 %. Déplacer des couches en cours de route, en revanche, n'est pas démontré.

Douze réponses, pas une moyenne

Un « 2,12× » est une médiane. Derrière, il y a douze réponses mesurées : trois processus sans correctif et trois avec, chacun interrogé avec une réponse de 128 puis de 512 jetons. Les deux nuages ne se touchent jamais : la plus lente des réponses corrigées (19,48 jetons/s) reste deux fois plus rapide que la plus rapide des autres (10,12).



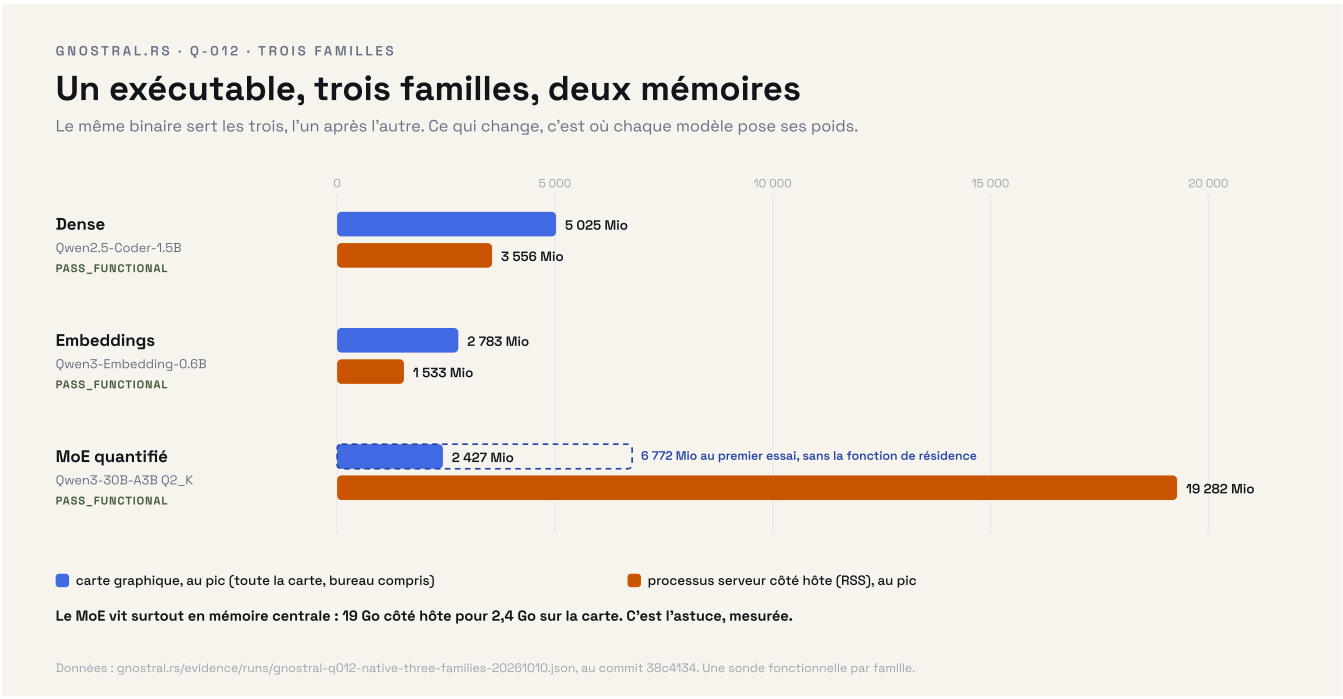
L'écart ne tient pas à une réponse chanceuse. Les textes produits sont identiques octet pour octet dans les six paires : le correctif accélère sans rien changer à ce qui est écrit.

› LES DONNÉES, RÉPONSE PAR RÉPONSE

Un exécutable, trois familles de modèles

Le jalon Q-012 pose une autre question : le même exécutable sait-il servir des modèles de natures différentes ? Trois familles, l'une après l'autre : un modèle dense qui écrit du texte, un modèle d'*embeddings* qui transforme des phrases en vecteurs, et le gros MoE quantifié. Les trois répondent juste — le modèle d'embeddings, par exemple, juge deux phrases liées bien plus proches (0,78) que deux phrases sans rapport (0,17).

Le graphique montre surtout **où chaque modèle pose ses poids**. Le MoE de 30 milliards de paramètres occupe 2,4 Go sur la carte et 19 Go en mémoire centrale : c'est l'astuce du dossier, mesurée. Le premier essai, compilé sans la fonction de résidence d'experts, montait à 6,8 Go sur la carte et a dépassé son délai ; il reste au dossier, et un garde refuse désormais un binaire qui l'oublie.



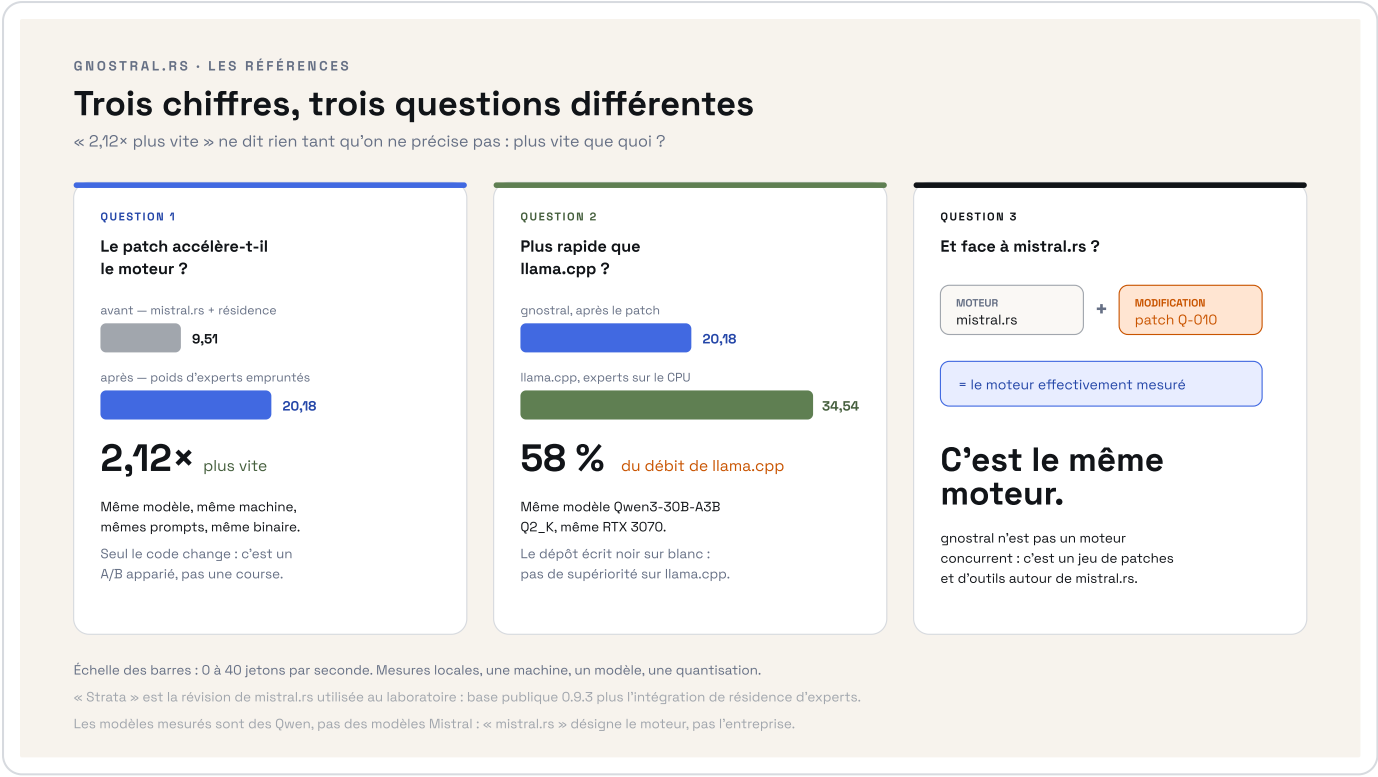
Deux mémoires, deux usages. La mémoire de la carte est mesurée pour la carte entière, bureau compris ; la mémoire hôte est celle du seul processus serveur. Une sonde fonctionnelle par famille : ce n'est pas un banc de qualité.

À RETENIR

Chaque chiffre a son rapport, son protocole et ses limites. Le dépôt refuse explicitement de publier un chiffre sans la preuve qui va avec — y compris quand la preuve contredit une affirmation plus flatteuse.

Par rapport à quoi, exactement ?

Un « 2,12× plus vite » ne veut rien dire sans son point de comparaison. Ici, la référence n'est pas un autre logiciel : c'est **le même moteur, une version plus tôt**.



Trois références, trois réponses. Le gain de 2,12× est un A/B apparié sur le même moteur. La comparaison avec llama.cpp est défavorable. La comparaison avec mistral.rs n'a pas de sens : c'est le moteur que le patch modifie.

Concrètement, « Strata » est la révision de mistral.rs utilisée au laboratoire : la version publique 0.9.3, plus une intégration de résidence d'experts. Les 9,51 jetons/s mesurent ce moteur *avant* le patch ; les 20,18 mesurent le même binaire, recompilé avec une seule fonction changée — les poids d'experts quantifiés sont empruntés au lieu d'être recopiés à chaque appel.

Face à llama.cpp, en revanche, la comparaison existe et elle est défavorable : **34,54 jetons/s pour llama.cpp contre 20,18 pour gnostral**, sur le même modèle et la même carte. Le rapport du dépôt l'écrit explicitement : ces gains n'établissent pas une supériorité sur les options CPU-MoE de llama.cpp.

Quant à mistral.rs, ce n'est pas un adversaire : c'est le moteur *dans lequel* le patch est appliqué. gnostral n'est pas un moteur concurrent — c'est un ensemble de patches, de contrats typés et d'outils de mesure autour de mistral.rs. Se comparer à lui reviendrait à comparer une voiture à son propre moteur.

ATTENTION AUX MOTS

mistral.rs est un moteur d'inférence écrit en Rust. **Mistral AI** est une entreprise qui produit des modèles. Les deux n'ont rien à voir, et les modèles mesurés ici sont des **Qwen**, pas des modèles Mistral.

Dire « plus rapide que Mistral » n'a donc pas de sens tant qu'on ne précise pas duquel on parle — et, s'il s'agit du moteur, quelle version sert de référence.

CE QUE ÇA CHANGE

La formulation exacte est : « 2,12× plus vite que la même révision de mistral.rs sans le patch, et encore 58 % du débit de llama.cpp sur ce modèle ». C'est moins spectaculaire, mais c'est vrai, daté et vérifiable.

Comment une mesure devient une preuve

La contribution la plus solide du projet n'est peut-être pas le gain de vitesse, mais la **méthode**. Six étapes, toujours les mêmes, et aucune ne peut être sautée.



La boucle de qualification. Un « PASS » ne dit que ce que le test a testé. Il ne dit rien de la stabilité, de la sécurité ou de la performance sur une autre machine.

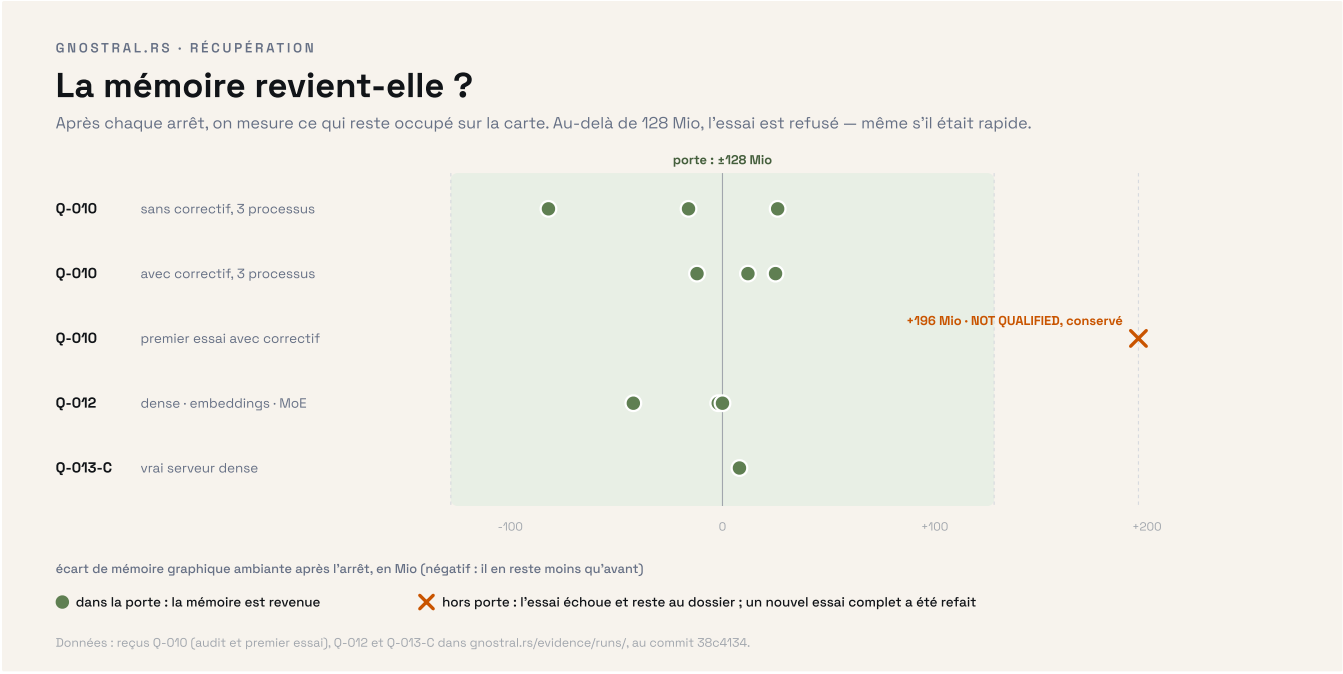
Concrètement : on identifie le code et le modèle par une empreinte (SHA), on écrit un plan qui décrit exactement ce qui sera exécuté, on lance un seul processus dans un cadre limité, on vérifie une réponse précise, on arrête, on vérifie que la mémoire est revenue, puis un programme indépendant refait les contrôles.

L'ordre compte. Un serveur peut répondre à une requête de contrôle alors que les poids ne sont pas encore chargés. gnostral.rs distingue donc huit états, du simple fait « déclaré dans la documentation » jusqu'à « qualifié par un audit indépendant ».

› LES HUIT ÉTATS DE LA VÉRITÉ, EN CLAIR

La mémoire revient-elle ?

La sixième étape de la boucle — l'arrêt et la récupération — est celle qui fait échouer le plus d'essais. Après chaque arrêt, on mesure ce qui reste occupé sur la carte graphique. La porte est fixée *avant* l'essai : **±128 Mio.** Un essai rapide qui laisse de la mémoire derrière lui est refusé.



Un échec visible au milieu des réussites. Le premier essai corrigé de Q-010 laissait 196 Mio sur la carte — cause non déterminée, peut-être le trafic du bureau. Il est marqué NOT QUALIFIED et conservé ; la mesure publiée vient d'une nouvelle session complète, aux mêmes seuils.

Ce que ce n'est pas

LES LIMITES, SANS DÉTOUR

- **Pas un serveur prêt à déployer.** Ce n'est pas un remplacement autonome de mistral.rs, llama.cpp, Candle ou vLLM.
- **Pas de changement de modèle à chaud.** La séquence A → B → A sous un même superviseur n'a pas été démontrée.
- **Pas d'API publique sécurisée.** Le pilote parle à localhost sur un port fixe, sans authentification d'un pair hostile.
- **Pas de service multi-utilisateur.** Aucun plan de contrôle ni engagement de disponibilité n'est ratifié.
- **Pas de garantie sur votre matériel.** Les chiffres viennent d'une configuration précise et sont explicitement non portables.

Pourquoi les échecs sont publics

Le premier essai de Q-010 a échoué : la mémoire graphique n'était pas revenue dans la marge attendue (+196 Mio contre une tolérance de ±128 Mio). Le premier essai de Q-012 a

échoué aussi : une fonctionnalité de compilation manquait, et le modèle MoE a dépassé le délai.

Les deux échecs sont toujours dans le dossier. C'est un choix méthodologique, pas une coquetterie : un résultat dont on ne connaît pas les ratés n'est pas réutilisable. Et un chiffre qui ne peut pas être contredit n'est pas une mesure, c'est une affirmation.

Le même principe s'applique au benchmark Bonsai : la référence retenue est 35,2 jetons par seconde, pas la revendication plus flatteuse de 40 — faute de preuve suffisante.

Historique : quinze quêtes en quinze jours

gnostral.rs avance par **quêtes** : chacune ouvre une question précise et se ferme par une décision écrite, y compris quand la décision est « non qualifié ». Le journal public en compte quinze entre le 7 et le 21 octobre 2026.



Deux journées font l'essentiel. Le 7 octobre ferme la recherche fondatrice et publie les versions 0.1 à 0.8 ; le 10, la chaîne de preuve native, de l'A/B jusqu'au vrai serveur supervisé.

Les projets étudiés, et refusés

Une part du travail consiste à examiner d'autres moteurs avant d'y toucher. Le dépôt garde les refus aussi soigneusement que les adoptions :

xInfer — non qualifié

Échec de compilation propre contre CUDA 13.4, chargements de modèles à court de mémoire. Adoption refusée : le laboratoire ne devient pas un atelier de réparation d'un projet amont.

zLLM — bloqué

Un modèle Gemma servi correctement, mais la promesse de placement adaptatif n'a pas pu être testée : le téléchargement de 17 à 19 Go aurait mangé près de la moitié de la marge disque.

UniLLM — étudié

Ses idées de registre et de cycle de vie sont reprises ; sa télémétrie simulée est rejetée. Règle qui en sort : écrire « inconnu » plutôt qu'une valeur plausible.

Une affirmation corrigée

La toute première quête, Q-001, n'a rien mesuré de nouveau : elle a **retiré** un chiffre. Un banc Bonsai annonçait 40 jetons par seconde ; faute de preuve suffisante, le dépôt a conservé 35,2 et n'a jamais publié le 40. Le projet commence donc par une correction — c'est un choix de méthode.

Projections : la feuille de route

La cible est écrite en une phrase : un plan de contrôle d'inférence **locale**, inspectable et écrit d'abord en Rust, pour du matériel contraint — choisir le bon moteur pour chaque modèle, borner les ressources, prouver qu'il est prêt, l'arrêter proprement, rendre compte de sa performance. Les noyaux de calcul et le placement des experts restent l'affaire de chaque moteur.



Une condition de sortie par jalon. M1 doit prouver qu'on parle bien au bon processus pendant tout le service ; M2, qu'on peut charger un modèle, le décharger, en charger un autre et revenir sans fuite de mémoire. Le service multi-modèle n'arrive qu'au bout, et seulement si tout le reste tient.

La recherche, en parallèle

Une veille à cinq pistes — transport de tenseurs quantifiés (Burn), métriques de latence côté client (Ferrum), cache KV réversible (WakeKV), plusieurs cartes (CubeCL), noyaux basse précision (Bonsai). Chacune a sa première expérience falsifiable ; aucune n'est une dépendance, et aucune n'a le droit de modifier en silence le moteur épinglé.

Ce qui ne sera pas promis

Pas d'API hébergée publique, pas d'affirmation de performance de production, pas de portabilité générale tant que la porte correspondante n'est pas passée. Un essai raté reste une preuve de premier rang, même quand un nouvel essai réussit.

Ce que ça change, selon qui vous êtes

Curieux de l'IA locale

Vous comprenez pourquoi un modèle ne « rentre » pas, et ce que coûte chaque choix de placement. Les analogies du dossier suffisent ; le dépôt fournit les détails.

Développeur

Le dépôt contient un contrat de moteur typé, un harnais de processus borné, des plans liés à une empreinte et des reçus de mesure en JSON. Beaucoup sont testables sans GPU.

Décideur ou financeur

L'actif n'est pas un produit fini : c'est une discipline de preuve, appliquée à un problème que la plupart des équipes préfèrent contourner en achetant du matériel.

Pour aller plus loin

> TECHNIQUE — LES MOTS DU DÉPÔT

> VÉRIFIER SANS CARTE GRAPHIQUE